

Rabbit R1 Boot Integrity, Recovery, and Ubuntu Touch

A practical textbook for MediaTek preloader mode, fastboot, A/B slots, Android Verified Boot, dm-verity, stock recovery, and the UBports installer

Prepared from the supplied Rabbit R1 terminal transcript

Research snapshot: 2026-08-07

Contents

Preface	6
How to use this book	6
1 The one-page answer	7
1.1 What happened	7
1.2 The recommended route from here	7
1.3 What not to do next	8
2 Part I - The system you are modifying	9
3 The Rabbit R1 boot chain	10
3.1 A layered boot, not one program	10
3.2 Why fastboot could be locked while mtkclient could write	11
3.3 Normal warning colors	11
4 Partitions, A/B slots, and super	12
4.1 Physical partitions and logical partitions	12
4.2 A/B boot slots	13
4.3 Why boot_1 failed	13
5 Android Verified Boot and dm-verity	14
5.1 AVB is the signed manifest; dm-verity is the block checker	14
5.2 What the fastboot disable flags actually do	14
5.3 Managed dm-verity and persistent EIO state	15
6 USB modes and the tools that see them	16
6.1 The four modes relevant here	16
6.2 cdc_acm in your log	16
6.3 fastboot versus fastbootd	17
7 Part II - Reading your terminal transcript	18
8 A forensic timeline of the commands	19
8.1 Stage 1: Preloader access worked	19
8.2 Stage 2: Fastboot accurately reported a locked bootloader	19
8.3 Stage 3: r1_escape changed the OEM-unlock policy state	20
8.4 Stage 4: sudo changed Python interpreters	21
8.5 Stage 5: The bootloader really became unlocked	21
8.6 Stage 6: fastboot -w partially succeeded	22
8.7 Stage 7: The first vbmeta command modified only the current slot	22

8.8	Stage 8: The intended GSI was never present	23
8.9	Stage 9: reboot-fastboot did not remain available	23
8.10	Stage 10: You restored stock boot images, but not a fully coherent stock state	24
9	The error messages translated	25
9.1	ModuleNotFoundError: No module named 'serial'	25
9.2	ModuleNotFoundError: No module named 'usb' only under sudo	25
9.3	externally-managed-environment	25
9.4	not allowed in locked state	25
9.5	Unlock operation is not allowed and unlock_ability = 0	25
9.6	unknown command for fastboot oem unlock	26
9.7	This partition doesn't exist for boot_1	26
9.8	Cannot generate image for userdata	26
9.9	No system.img	26
9.10	< waiting for any device > after rebooting to fastbootd	26
10	Current-state assessment	27
10.1	Facts established by the log	27
10.2	Likely, but not proven from the transcript alone	27
10.3	Unknown and worth measuring	27
10.4	A practical state matrix	28
11	Part III - What Ubuntu Touch is on this device	29
12	Ubuntu Touch, Halium, and the Rabbit port	30
12.1	Ubuntu Touch is not desktop Ubuntu copied onto a phone	30
12.2	Why stock rabbitOS v0.8 is a prerequisite	30
12.3	Current support status and realistic expectations	31
12.4	No supported dual boot	31
13	What the UBports Installer actually does	32
13.1	Device-specific button actions	32
13.2	Installer prerequisites	32
13.3	Downloaded bootstrap images	33
13.4	Flash order	33
13.5	Why this can repair an ordinary mismatch	34
14	Part IV - Recovery and installation runbook	35
15	Safety preparation	36
15.1	Stop conditions	36
15.2	Host checklist	36
15.3	Use one Python environment	37
15.4	Preserve the firmware archive and hashes	37
15.5	Back up critical persistent partitions	38
15.6	Enter bootloader fastboot cleanly	38
15.7	Capture the preflight state	38
16	Phase 1 - Restore a coherent stock base	40
16.1	Preferred method: Rabbit's official WebUSB flash tool	40
16.2	What a stock restore should and should not do	40

16.3 Manual restore is a fallback, not the first choice	41
17 Phase 2 - Normalize A/B state	42
17.1 Inspect before erasing metadata	42
17.2 Rabbit-specific metadata reset	42
17.3 Attempt one stock boot	43
18 Phase 3 - The seccfg decision gate	44
18.1 Enter this phase only when all conditions are true	44
18.2 What the community recovery found	44
18.3 Why a hex editor alone is unsafe	45
18.4 Required verification before any write	45
18.5 Use the source procedure, not an abbreviated copy	45
19 Phase 4 - Install Ubuntu Touch	47
19.1 Establish the prerequisite state	47
19.2 Obtain and run the official installer	47
19.3 Installer selections for this device	48
19.4 During the bootstrap	48
19.5 First boot	49
20 Phase 5 - Validate the installation	50
20.1 Basic functional checks	50
20.2 Expected limitations	50
20.3 Preserve a recovery notebook	51
21 Phase 6 - Return to stock if needed	52
21.1 Preferred rollback	52
21.2 Why gsi wipe is not the rollback plan	52
22 Part V - Diagnostic reasoning and field reference	53
23 The recovery decision tree	54
24 May the installer be tried immediately?	56
25 Installer-stage troubleshooting	57
26 Why common alternative approaches fail	58
26.1 Flashing only boot.img	58
26.2 Flashing only vbmeta.img	58
26.3 Flashing a GSI system.img without a stable fastbootd	58
26.4 Relocking to “make verification normal”	58
26.5 Erasing everything MediaTek exposes	59
27 Command semantics reference	60
27.1 MediaTek and serial commands	60
27.1.1 python mtkbootcmd.py FASTBOOT	60
27.1.2 mtk printgpt	60
27.1.3 mtk r PARTITION FILE	60
27.1.4 mtk w PARTITION FILE	60

27.2	Fastboot state commands	60
27.2.1	fastboot devices -l	60
27.2.2	fastboot getvar NAME	61
27.2.3	fastboot flashing get_unlock_ability	61
27.2.4	fastboot flashing unlock	61
27.2.5	fastboot set_active a	61
27.3	Fastboot write commands	61
27.3.1	fastboot flash boot_a boot.img	61
27.3.2	fastboot flash vbmeta_a vbmeta.img	61
27.3.3	fastboot --disable-verity --disable-verification flash vbmeta_a vbmeta.img	61
27.3.4	fastboot flash super super.img	61
27.3.5	fastboot format userdata	61
27.3.6	fastboot -w	62
27.4	Reboot commands	62
27.4.1	fastboot reboot	62
27.4.2	fastboot reboot bootloader	62
27.4.3	fastboot reboot-fastboot	62
28	A compact command sheet for your next session	63
28.1	1. Set paths and environment	63
28.2	2. Back up persistent state	63
28.3	3. Enter bootloader fastboot and record state	64
28.4	4. Restore stock with the official Rabbit tool	64
28.5	5. Select and inspect slot A	64
28.6	6. Test stock	65
28.7	7. Run UBports Installer	65
29	Concept checks	66
29.1	Why did fastboot devices initially show nothing even though FASTBOOT was sent?	66
29.2	Why did changing FRP affect unlock ability?	66
29.3	Why is a stock super.img not the same thing as the GSI's system.img?	66
29.4	Why can an unlocked device still show dm-verity corruption?	66
29.5	Why restore stock before Ubuntu Touch if the installer overwrites the main images?	67
29.6	Why leave the bootloader unlocked after Ubuntu Touch boots?	67
30	Glossary	68
31	Selected source notes	70

Preface

This book explains the boot failure shown in the supplied terminal transcript and gives a controlled route from that state to Ubuntu Touch on a Rabbit R1. It is written for a Linux host and assumes comfort with a terminal, USB device modes, hashes, and destructive storage operations.

Risk notice. Unlocking and flashing erase user data. Raw MediaTek writes can permanently damage boot, radio calibration, or device identity data. The conservative order is: collect evidence, preserve backups, restore a matched stock image set, verify slot state, and only then consider the `seccfg` procedure. Never relock the bootloader while modified or verification-disabled images are installed.

The document distinguishes three very different problems that can produce similar screens:

1. a mismatched set of `boot`, `vbmeta`, and `super` images;
2. A/B metadata that has marked one or both slots unbootable; and
3. a persistent MediaTek bootloader dm-verity state stored outside the ordinary Android images.

The official Rabbit flash tool and the official UBports Installer should be preferred over manual partition-by-partition flashing. The community `seccfg` method is a last-resort recovery technique, not a routine Ubuntu Touch installation step.¹

How to use this book

Read Chapter 1 first. It gives the shortest safe path for the specific device state in the transcript. Chapters 2 through 8 explain the architecture and reconstruct what each command did. Chapters 9 through 14 are the recovery and installation runbook. The appendices provide compact command sheets, an error dictionary, and source notes.

Identifiers from the supplied log, including the fastboot serial, Bluetooth address, ME ID, and SoC ID, are intentionally omitted. Device owners should redact those values before posting logs publicly.

¹The community recovery explicitly recommends official Rabbit tools first, describes raw `seccfg` work as a last resort, and warns not to relock with disabled `vbmeta` installed. See source note 13.

The one-page answer

1.1 What happened

The unlock bypass worked. At first, fastboot rejected writes because the device was locked and reported `unlock_ability = 0`. The `r1_escape` process then used `mtkclient` to read the `frp` partition, changed the script-selected byte, wrote the partition back, and returned to fastboot. After that, fastboot flashing unlock completed successfully. The later successful writes to `boot_a`, `boot_b`, and the `vbmata` partitions confirm that the bootloader was no longer rejecting flashes.

The intended `r1_escape` installation did **not** complete. That project expects a separately downloaded Android GSI named `system.img`; your `find . -name system.img` returned nothing. The stock Rabbit firmware contains `super.img`, not a standalone `system.img`, because modern Android stores `system`, `vendor`, `product`, and related logical partitions inside the physical `super` partition. The script therefore stopped after unlock, data erasure, and `vbmata` modification, leaving no complete replacement operating system.¹²

You then restored stock `boot.img` to both A/B slots and flashed stock `vbmata`-family files with `--disable-verity --disable-verification`. That is not a fully stock verified-boot state: the image payload came from stock, but fastboot rewrote its verification flags. Repeated boot attempts and slot changes may also have consumed A/B retries. The red `dm-verity corruption` screen means that the bootloader is entering AVB's `eio` error mode; an orange warning alone merely means the bootloader is unlocked.³

1.2 The recommended route from here

1. **Stop relocking and stop mixing individual images.** Keep the bootloader unlocked.
2. **Record the current fastboot variables and back up critical MediaTek partitions.** At minimum, preserve `seccfg` and `frp`; preserve `radio`/`calibration` partitions if they exist in the GPT, but never erase or rewrite them casually.

¹The `r1_escape` README states that its script enables OEM unlocking, unlocks the bootloader, disables AVB, and flashes an AOSP 13 userdebug GSI. Its script expects a separately provided `system.img`. See source note 12.

²AOSP documents that `super` contains dynamic-partition metadata and logical partitions, and that a factory `super.img` bundles `system/vendor/etc.` and may be flashed directly without fastbootd. See source note 5.

³AOSP's reference boot flow defines orange for unlocked devices and red `eio` for `dm-verity` corruption. See source note 1.

3. **Restore one coherent stock rabbitOS v0.8.293 set using Rabbit’s official WebUSB flash tool.** Do not add disable-verification flags to the stock restore. Do not relock afterward.⁴
4. **Check and normalize A/B slot metadata.** Select slot A and confirm that it is not marked unbootable. Use the Rabbit-specific `boot_para/para` reset only if a full matched stock flash still leaves bad slot metadata.
5. **Boot stock once.** If stock reaches setup, the persistent verity state is clear. Return to fastboot and run the UBports Installer.
6. **If a full stock restore still gives the same red dm-verity screen, treat `seccfg` as the remaining suspect.** Follow the community procedure only after backing up the original partition and verifying the generated V4 header. The corrected state described in that recovery has `lock_state = 3`, the word at offset `0x10` set to `0`, and the managed-verity record at `0x240:0x2c0` cleared.⁵
7. **Install Ubuntu Touch with the official UBports Installer.** Select Rabbit R1, enable **Bootstrap**, enable **Wipe Userdata**, and choose the stable channel currently offered by the installer. The installer downloads checksummed `boot.img`, `super.img`, and `vbmeta.img`, flashes them from bootloader fastboot, formats `userdata`, enters UBports recovery, and installs the system image. It does not require your broken `fastbootd` transition.⁶
8. **Leave the bootloader unlocked.** The orange warning on every boot is expected.

1.3 What not to do next

- Do not run `fastboot flashing lock` or any OEM relock command.
- Do not keep reflashing `vbmeta` with different flag combinations.
- Do not erase `nvr`, `nvdata`, `proinfo`, `protect1`, `protect2`, or calibration partitions.
- Do not assume `fastboot -w` failed completely: your log shows `userdata` was erased before the host-side formatter failed.
- Do not chase a missing stock `system.img`; stock logical partitions are inside `super.img`.
- Do not keep multiple serial watchers or `mtkclient` sessions open while the device is changing USB modes.

⁴Rabbit’s official WebUSB tool instructs users to download the firmware release, enter preloader fastboot, select the fastboot device, and flash the stock ROM folder. See source notes 10 and 11.

⁵The community recovery reports a Rabbit-specific persistent dm-verity state in V4 `seccfg`, with lock state at `0x0c`, a critical/dm-verity field at `0x10`, and a managed-verity record at `0x240:0x2c0`. See source note 13.

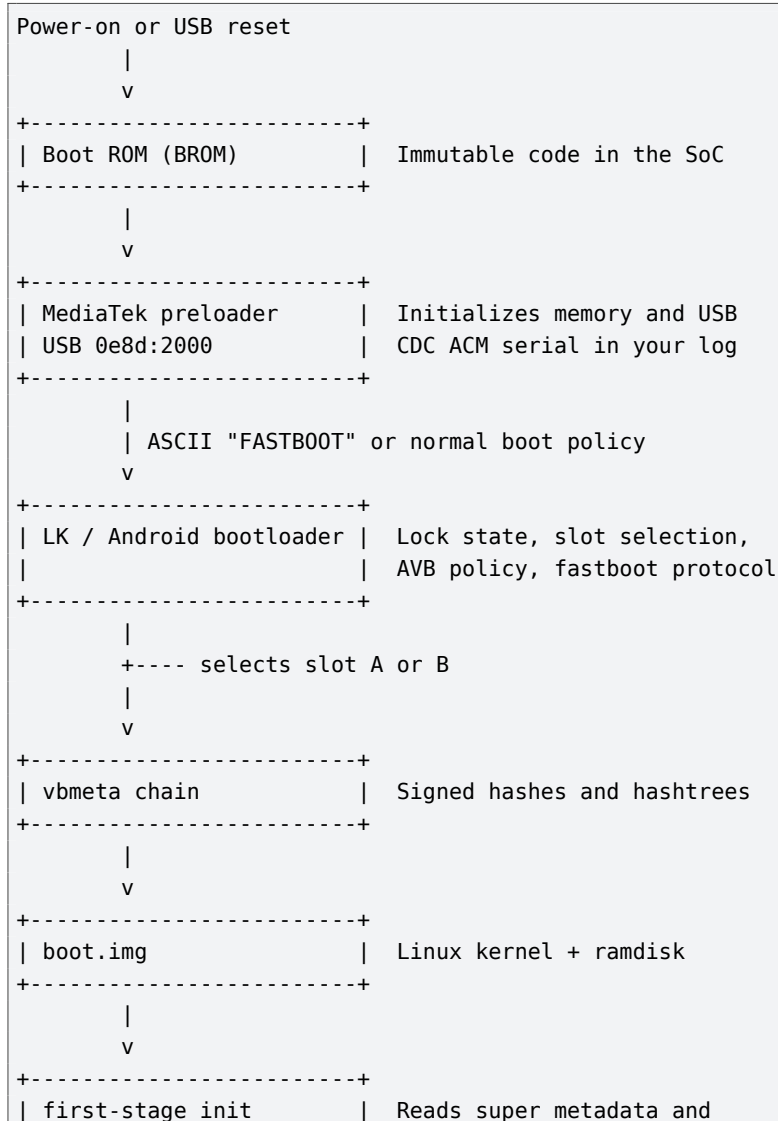
⁶The official UBports installer configuration for `r1` requires stock rabbitOS v0.8 and an unlocked bootloader, downloads checksum-pinned `boot/super/vbmeta` artifacts, flashes those physical partitions, formats `userdata`, and reboots to UBports recovery. See source note 8.

Part I - The system you are modifying

The Rabbit R1 boot chain

3.1 A layered boot, not one program

The Rabbit R1 uses a MediaTek MT6765-family platform. Several independent components run before Android or Ubuntu Touch appears:





A failure at each layer looks different. `mtkbootcmd.py` only asks the preloader to enter bootloader fastboot. It does not unlock anything and it does not repair AVB. `mtkclient` works below fastboot and can access raw eMMC through MediaTek's Download Agent path. Fastboot works in LK and obeys LK's lock policy. Android Verified Boot begins when LK selects and verifies a boot slot.

3.2 Why fastboot could be locked while mtkclient could write

The transcript shows `mtkclient` reporting that the target was unprotected and that memory read/write authentication was not required. That allowed a Download Agent to read and write the raw `frp` partition. Fastboot, however, initially returned not allowed in locked state. These statements are not contradictory: the two tools use different protocols at different layers.

A useful mental model is:

mtkclient authority:	BROM/preloader/Download Agent -> raw storage
fastboot authority:	LK bootloader -> policy-controlled partitions
ADB authority:	running Android/Ubuntu userspace -> OS services

The fact that one path works says little about another path.

3.3 Normal warning colors

Android's reference boot flow defines an orange warning for an unlocked device and a red `eio` warning for dm-verity corruption. An unlocked device can show orange and still boot normally. A red `eio` screen means the bootloader found an OS but is reporting a hashtree verification error mode to the kernel.¹

Therefore:

- **Orange only:** expected after unlock and expected with Ubuntu Touch.
- **Red dm-verity corruption:** not merely an unlock warning; diagnose image consistency, slot state, and persistent managed-verity state.
- **No valid OS / signature error:** usually a more fundamental boot-chain or lock-state mismatch.

¹AOSP's reference boot flow defines orange for unlocked devices and red `eio` for dm-verity corruption. See source note 1.

Partitions, A/B slots, and super

4.1 Physical partitions and logical partitions

Modern Android devices combine old-style physical GPT partitions with dynamic logical partitions.

Physical GPT partitions (selected examples)

boot_a	boot_b	kernel and ramdisk for each slot
vbmeta_a	vbmeta_b	top-level AVB metadata
vbmeta_system_a/b		chained system AVB metadata
vbmeta_vendor_a/b		chained vendor AVB metadata
dtbo_a	dtbo_b	device-tree overlays
lk_a	lk_b	bootloader stage
tee_a	tee_b	trusted execution environment
super		container for dynamic partitions
userdata		mutable user data
metadata		encryption/update metadata
frp		factory reset / OEM-unlock policy data
seccfg		MediaTek security state
nvram, nvdata, protect*		identity, radio, or calibration data

Inside `super`, Android creates logical devices such as:

```
super
+-- system_a / system_b
+-- vendor_a / vendor_b
+-- product_a / product_b
+-- system_ext_a / system_ext_b
+-- odm_a / odm_b, if used
```

The exact logical layout is stored in `super` metadata. This is why the Rabbit stock archive can contain one large `super.img` instead of separate `system.img`, `vendor.img`, and `product.img` files. A factory `super.img` bundles the logical-partition content and can be flashed directly to the physical `super` partition without entering fastbootd.¹

¹AOSP documents that `super` contains dynamic-partition metadata and logical partitions, and that a factory `super.img` bundles `system/vendor/etc.` and may be flashed directly without fastbootd. See source note 5.

4.2 A/B boot slots

A/B devices maintain two bootable sets so an update can be written to the inactive slot and rolled back if it fails. The bootloader stores metadata for each slot:

- active/current slot;
- successful flag;
- unbootable flag;
- remaining retry count.

Fastboot defaults to the current slot when a partition name has an A/B suffix but the command omits it. This explains why your unsuffixed command first reported Sending 'vbmeta_b' and later reported Sending 'vbmeta_a': the active slot had changed. The AOSP fastboot interface exposes `current-slot`, `slot-successful:<slot>`, `slot-unbootable:<slot>`, and `slot-retry-count:<slot>`; selecting a slot should clear its unbootable flag and reset retries.²

Repeated failed boots can exhaust retry counts. The bootloader then falls back to the other slot. If both slots are invalid or marked unbootable, the device may bounce between the logo, preloader enumeration, and power-off without ever starting a userspace that provides ADB or fastbootd.

4.3 Why boot_1 failed

Android slot suffixes are `_a` and `_b`, not `_1` and `_2`. The failure:

```
Writing 'boot_1' FAILED (remote: 'This partition doesn't exist')
```

was literal. The correct names are `boot_a` and `boot_b`, which then flashed successfully.

²AOSP documents default-current-slot fastboot behavior and the slot variables for successful, unbootable, and retry state. See source note 4.

Android Verified Boot and dm-verity

5.1 AVB is the signed manifest; dm-verity is the block checker

Android Verified Boot 2.0 uses signed vbmeta structures to describe what must be trusted. A top-level vbmeta image can contain hashes for small partitions and hashtree descriptors for large read-only filesystems. It can also delegate verification to chained metadata such as vbmeta_system and vbmeta_vendor. Android's reference AVB tooling can verify a signed vbmeta object, a boot hash, a system hashtree, and chained partition keys as one trust graph.¹

dm-verity is the kernel device-mapper target that checks filesystem blocks against a Merkle hashtree while they are read. In simplified form:

```
signed vbmeta
|
+-- hash(boot.img)
+-- chain(vbmeta_system)
|   +-- hashtree(system)
|   +-- hashtree(product)
+-- chain(vbmeta_vendor)
|   +-- hashtree(vendor)
```

A complete image set must agree across this graph. Flashing a stock boot.img with a vbmeta generated for another build, or flashing a new super.img while leaving old chained metadata, can trigger verification errors even though every individual file is valid by itself.

5.2 What the fastboot disable flags actually do

When used while flashing a vbmeta image:

```
fastboot --disable-verity --disable-verification flash vbmeta vbmeta.img
```

fastboot rewrites flags in the vbmeta structure before writing it. Conceptually:

- `--disable-verification` disables descriptor/signature enforcement for the chain rooted there;
- `--disable-verity` disables AVB hashtree enforcement.

¹AOSP and the AVB source describe signed vbmeta descriptors, partition hashes, hashtrees, and chained partitions. See source notes 2 and 6.

These flags are development tools. They do not unlock the bootloader. They are meaningful for flash vbmata ..., not for fastboot flashing unlock. The command:

```
fastboot flashing unlock --disable-verification
```

therefore does not make sense: unlock policy and vbmata flags are separate mechanisms.

5.3 Managed dm-verity and persistent EIO state

AVB supports a managed error mode that can persist state outside ordinary boot images. The reference implementation names its persistent value `avb.managed_verity_mode` and maps the managed restart/EIO mode to kernel parameters including `androidboot.veritymode=eio` and `androidboot.veritymode.managed=yes`.²

The Rabbit community recovery found an additional MediaTek-specific representation in `seccfg`: a V4 header word at offset `0x10` remained 1 while `lock_state` at `0x0c` was 3 (unlocked). LK treated this as a persistent dm-verity error state and regenerated the managed-verity record after ordinary reflashes. Clearing only Android images therefore did not clear the red screen.³

This is why a full stock reflash is the diagnostic boundary:

- If a matched stock set boots, the problem was image/slot consistency.
- If a matched stock set verifies but the bootloader still emits `veritymode=eio ... managed=yes`, investigate `seccfg`.

Do not jump directly to `seccfg` merely because the screen says dm-verity. Most dm-verity failures are fixed by restoring a coherent chain.

²The AVB source documents persistent storage named `avb.managed_verity_mode` and the mapping of managed EIO mode to `androidboot.veritymode` parameters. See source note 6.

³The community recovery reports a Rabbit-specific persistent dm-verity state in V4 `seccfg`, with lock state at `0x0c`, a critical/dm-verity field at `0x10`, and a managed-verity record at `0x240:0x2c0`. See source note 13.

USB modes and the tools that see them

6.1 The four modes relevant here

Mode	Typical host interface	Main tool	Purpose
BROM	MediaTek USB device	mtkclient	Earliest SoC recovery path
Preloader	0e8d:2000, CDC ACM, /dev/ttyACM0	mtkbootcmd.py, mtkclient	Initialize hardware; request fastboot or load DA
Bootloader fastboot	MediaTek fastboot USB PID	fastboot	Flash physical partitions, unlock, choose slot
OS/recovery userspace	ADB/MTP, or fastbootd	adb, UBports Installer, sometimes fastboot	Install system image, manipulate logical partitions

The USB device disconnecting and reappearing is expected. Each transition changes USB descriptors and often the Linux driver binding.

6.2 cdc_acm in your log

mtkbootcmd.py imports pyserial, waits for a MediaTek preloader CDC ACM port, opens /dev/ttyACM0, and writes FASTBOOT. The Linux cdc_acm driver is therefore required for that step. After the R1 re-enumerates as bootloader fastboot, the serial port disappears and fastboot uses a different USB interface. There is no benefit to repeatedly loading or unloading cdc_acm while the device is already in fastboot.

Some community Android instructions unload cdc_acm later to avoid a host-specific ADB binding conflict. Treat that as a mode-specific workaround, not as a universal requirement. For your immediate recovery, load it for the preloader-to-fastboot transition and leave module blacklisting alone until the operating system is booting.

6.3 fastboot versus fastbootd

Bootloader fastboot runs in LK and can flash physical partitions such as `boot`, `vbmata`, and a factory `super.img`. Fastbootd is a userspace implementation launched from recovery so it can understand and resize logical partitions inside `super`.

The `r1_escape` script wanted fastbootd because it intended to flash one logical `system` partition from a GSI. Your device disappeared after `fastboot reboot-fastboot`, likely because the current boot/recovery path could not keep fastbootd alive. That failure does not block the official Ubuntu Touch route: the Rabbit R1 installer configuration flashes a complete factory-style `super.img` directly from bootloader fastboot.¹

¹The official UBports installer configuration for `r1` requires stock rabbitOS v0.8 and an unlocked bootloader, downloads checksum-pinned `boot/super/vbmata` artifacts, flashes those physical partitions, formats userdata, and reboots to UBports recovery. See source note 8.

Part II - Reading your terminal transcript

A forensic timeline of the commands

8.1 Stage 1: Preloader access worked

The first failure was purely a Python dependency problem:

```
ModuleNotFoundError: No module named 'serial'
```

Installing `pyserial` fixed the import. The subsequent output is significant:

```
Found /dev/ttyACM0 with description: MT65xx Preloader  
b'FASTBOOT' cmd sent
```

This proves that:

- the R1 was reaching MediaTek preloader mode;
- the cable and USB path could carry the preloader serial protocol;
- `mtkbootcmd.py` found the correct USB VID/PID;
- the script successfully transmitted its command.

It did **not** prove that bootloader fastboot had finished enumerating. The empty fastboot devices output immediately afterward can result from transition timing, the device leaving fastboot again, USB permissions, or a failing bootloader path. Later in the same transcript, the same procedure produced a visible fastboot serial, so the basic mechanism was sound.

8.2 Stage 2: Fastboot accurately reported a locked bootloader

These writes transferred the image to RAM but were rejected at the write-policy step:

```
Sending 'boot_a' ... OKAY  
Writing 'boot_a' ... FAILED (remote: 'not allowed in locked state')
```

Sending success only means the host-to-device transfer worked. Writing is the operation that modifies storage, and LK denied it.

The unlock diagnostics were also clear:

```
fastboot flashing get_unlock_ability
unlock_ability = 0

fastboot flashing unlock
Unlock operation is not allowed
```

The following experiments were dead ends:

- fastboot oem unlock: OEM-specific command not implemented;
- fastboot bootloader unlock: not valid fastboot syntax;
- get_unlock_capability: wrong command name; this client exposes get_unlock_ability;
- disable-verification flags appended to flashing unlock: vbmata options do not change unlock eligibility.

8.3 Stage 3: r1_escape changed the OEM-unlock policy state

The project describes its process as enabling OEM unlocking, unlocking the bootloader, disabling AVB, and flashing an AOSP 13 GSI.¹ Its shell script does the enabling step by reading the `frp` partition, changing the last byte from `00` to `01` when necessary, and writing it back.

Your first successful `mtkclient` session did the following:

```
Device is unprotected
Device is in Preloader-Mode
Successfully uploaded stage 1
Successfully uploaded stage 2
Dumping partition "frp"
```

You then changed the final byte and wrote the image back:

```
Wrote frp.bin to sector ...
```

After returning to fastboot:

```
fastboot flashing unlock
OKAY
```

The before-and-after sequence is strong evidence that the script's FRP modification enabled the fastboot unlock flow on this Rabbit build. It is also the point at which all user data became expendable: standardized Android unlocking is expected to wipe or require wiping user data.²

¹The `r1_escape` README states that its script enables OEM unlocking, unlocks the bootloader, disables AVB, and flashes an AOSP 13 userdebug GSI. Its script expects a separately provided `system.img`. See source note 12.

²AOSP distinguishes locked devices, which prevent modification, from unlocked devices, which allow it, and describes the standardized unlock transition. See source note 3.

8.4 Stage 4: sudo changed Python interpreters

The user Python environment had pyusb installed:

```
Requirement already satisfied: pyusb in  
/home/manuel/.pyenv/versions/3.13.2/...
```

But this command failed:

```
sudo python3 mtk ...  
ModuleNotFoundError: No module named 'usb'
```

`sudo python3` resolved to the distribution's system Python, not the pyenv interpreter into which dependencies were installed. Debian's PEP 668 protection then correctly refused a system-wide `sudo pip3 install`.

The non-sudo command succeeded because it used the pyenv interpreter. A cleaner pattern is to create one project virtual environment and invoke that exact interpreter even when root USB access is temporarily required:

```
R1ESC="$HOME/code/others/rabbit-r1/r1_escape"  
  
cd "$R1ESC"  
python3 -m venv .venv  
"$R1ESC/.venv/bin/python" -m pip install --upgrade pip  
"$R1ESC/.venv/bin/python" -m pip install pyserial  
"$R1ESC/.venv/bin/python" -m pip install -r "$R1ESC/mtkclient/requirements.txt"  
  
# Normal-user serial command:  
"$R1ESC/.venv/bin/python" "$R1ESC/mtkbootcmd.py" FASTBOOT  
  
# Only if raw USB permissions require root:  
sudo "$R1ESC/.venv/bin/python" "$R1ESC/mtkclient/mtk" printgpt
```

This avoids installing packages into the OS-managed Python and avoids the hidden interpreter switch.

8.5 Stage 5: The bootloader really became unlocked

After the FRP write, both of these facts appear:

```
fastboot flashing unlock ... OKAY  
fastboot flash boot_a ... OKAY
```

A successful partition write is practical confirmation that LK no longer considered the device locked for ordinary flashing. Re-running the unlock command later was unnecessary. The stable diagnostic would have been:

```
fastboot getvar unlocked 2>&1  
fastboot flashing get_unlock_ability 2>&1
```

The `unlock-ability` variable answers whether unlocking may be initiated; it is not always the best variable for the already-unlocked state.

8.6 Stage 6: `fastboot -w` partially succeeded

The relevant output was:

```
Erasing 'userdata' OKAY
mke2fs failed with status 1
fastboot: error: Cannot generate image for userdata
```

`fastboot -w` is a composite operation. In your run:

1. the bootloader erased `userdata` successfully;
2. the host `fastboot` package then tried to generate a filesystem image;
3. its bundled or invoked `mke2fs` failed.

This means data was likely gone even though the overall command returned an error. It does not mean unlock failed. Common causes include a distribution-packaged platform-tools bug, missing `e2fsprogs` support, an unsupported feature flag, or a size/format mismatch. The UBports Installer has its own `fastboot:format userdata` step, so repeatedly running `fastboot -w` is not the best next action.

8.7 Stage 7: The first `vbmeta` command modified only the current slot

This command:

```
fastboot flash --disable-verity --disable-verification vbmeta vbmeta.img
```

reported:

```
Sending 'vbmeta_b'
```

Later, the same unsuffixed command targeted `vbmeta_a`. That is normal A/B behavior: `fastboot` resolves a slotted base name to the current slot.³ It also means the device changed active slot at least once during the troubleshooting session.

The small `vbmeta.img` in the `r1_escape` repository was intended for the GSI path, not as a complete stock Rabbit restore. Later you flashed Rabbit's stock `vbmeta.img`, `vbmeta_system.img`, and `vbmeta_vendor.img` to both slots, but you asked `fastboot` to rewrite verification flags on each one. The result was internally more complete than a one-slot flash, yet it was still a development-mode AVB chain rather than an untouched stock chain.

³AOSP documents default-current-slot `fastboot` behavior and the slot variables for successful, unbootable, and retry state. See source note 4.

8.8 Stage 8: The intended GSI was never present

The script ends with:

```
fastboot reboot-fastboot
fastboot flash system system.img
fastboot reboot
```

Your working tree contained no `system.img`:

```
find . -name system.img
# no output
```

The Rabbit stock ZIP instead contained:

```
super.img
super_empty.img
boot.img
vbmeta.img
vbmeta_system.img
vbmeta_vendor.img
...
```

Therefore the `r1_escape` flow did not install its promised AOSP system. The device was left between states: unlocked and partly de-verified, with stock system content still in `super` unless some unshown operation changed it.

8.9 Stage 9: `reboot-fastboot` did not remain available

Several runs produced:

```
Rebooting into fastboot OKAY
< waiting for any device >
```

and the R1 later reappeared as preloader. This suggests that the recovery/userspace image needed to host `fastbootd` was not booting reliably. Possible contributing factors include:

- mismatched `boot` and `super` content;
- bad A/B selection or exhausted retries;
- a recovery-as-boot design that depends on the current `boot.img`;
- the persistent verity mode forcing an early failure;
- the device simply rebooting out of `fastbootd` before the host attached.

The important conclusion is limited: **fastbootd was not stable in that state**. It does not imply that the physical `super` partition cannot be flashed from bootloader `fastboot`, and it does not block the official Rabbit Ubuntu Touch installer.

8.10 Stage 10: You restored stock boot images, but not a fully coherent stock state

These commands succeeded:

```
fastboot flash boot_a stock/boot.img  
fastboot flash boot_b stock/boot.img
```

Then all six vbmeta-family partitions were flashed with disable flags. This made both slots more symmetrical, but other boot-chain components (lk, dtbo, tee, and the shared `super`) were not shown being restored in that pass. A coherent restore should be treated as one transaction: all relevant images must come from the same firmware package, and stock vbmeta files should be written without development flag rewriting.

The error messages translated

9.1 `ModuleNotFoundError: No module named 'serial'`

Meaning: the active Python interpreter lacked `pyserial`.

Correct response: install into the exact interpreter or virtual environment used to run `mtkbootcmd.py`.

9.2 `ModuleNotFoundError: No module named 'usb' only under sudo`

Meaning: `sudo` selected another Python installation that lacked `pyusb`.

Correct response: invoke the virtual environment's Python by absolute path, or configure USB permissions and avoid root.

9.3 `externally-managed-environment`

Meaning: the Linux distribution protects its system Python under PEP 668.

Correct response: use a virtual environment. Do not use `--break-system-packages` for this project.

9.4 `not allowed in locked state`

Meaning: transfer succeeded, but LK refused the storage write because the bootloader was locked.

Correct response: enable unlock eligibility, then use `fastboot flashing unlock`. Do not search for a `vbmata` flag workaround.

9.5 `Unlock operation is not allowed and unlock_ability = 0`

Meaning: the standardized unlock flow was disabled by persistent policy data.

Correct response on this device: the `r1_escape` FRP modification changed that policy, after which unlock succeeded. Do not repeat the write now that the device is unlocked.

9.6 unknown command for fastboot oem unlock

Meaning: the Rabbit bootloader does not implement that OEM command.

Correct response: use the standardized `fastboot flashing unlock` flow.

9.7 This partition doesn't exist for boot_1

Meaning: wrong partition name.

Correct response: use `boot_a` OR `boot_b`.

9.8 Cannot generate image for userdata

Meaning: host-side filesystem image generation failed after the erase.

Correct response: stop repeating wipes; use the official stock `userdata.img` when restoring stock, or let the UBports Installer format `userdata`.

9.9 No system.img

Meaning: the GSI required by `r1_escape` was never downloaded.

Correct response: do not substitute Rabbit's `super.img` into a command that expects a logical `system.img`. For Ubuntu Touch, use the official installer, which knows that its port image is a complete `super.img`.

9.10 < waiting for any device > after rebooting to fastbootd

Meaning: the current USB mode disappeared and the requested userspace `fastboot` did not remain reachable.

Correct response: re-enter bootloader `fastboot` through scroll-wheel-up plus power or the pre-loader serial command. Do not make `fastbootd` a prerequisite for the Rabbit Ubuntu Touch installation.

Current-state assessment

10.1 Facts established by the log

The following are high-confidence conclusions:

- The MediaTek preloader path works.
- `mtkclient` can communicate with the device in preloader mode.
- The FRP unlock-enablement step was written.
- `fastboot flashing unlock` succeeded.
- Bootloader fastboot can flash partitions.
- `userdata` was erased at least once.
- Stock `boot.img` was written to both slots.
- Both slots received stock-source vbmeta-family images with verification-disable flags.
- No GSI `system.img` was flashed by the shown commands.
- Fastbootd did not stay reachable.

10.2 Likely, but not proven from the transcript alone

- The original stock `super` content may still be present because no successful `system` or `super` flash is shown after unlock.
- One or both A/B slots may have consumed retry counts or been marked unbootable.
- The persistent `seccfg dm-verity` state may be set, especially if a full matched stock flash has already been tried and the red screen remains.
- The current active slot may change after each failed boot attempt.

10.3 Unknown and worth measuring

Before the next flash, collect this snapshot from bootloader fastboot:

```
mkdir -p rl-diagnostics

{
  date -Is
  fastboot --version
  fastboot devices -l
}
```

```

fastboot getvar product
fastboot getvar unlocked
fastboot getvar secure
fastboot getvar current-slot
fastboot getvar slot-count
fastboot getvar slot-successful:a
fastboot getvar slot-unbootable:a
fastboot getvar slot-retry-count:a
fastboot getvar slot-successful:b
fastboot getvar slot-unbootable:b
fastboot getvar slot-retry-count:b
fastboot getvar is-userspace
fastboot getvar has-slot:boot
fastboot getvar has-slot:vbmeta
fastboot getvar partition-size:super
} > r1-diagnostics/fastboot-state.txt 2>&1

```

Fastboot writes many variable responses to standard error, so 2>&1 is intentional.

Do not publish the output without removing serial numbers and hardware identifiers.

10.4 A practical state matrix

Observation after a matched stock restore	Most likely interpretation	Next action
Orange warning, then stock setup	Unlock is normal; AVB path can boot	Proceed to UBports Installer
Red dm-verity screen, then stock boots after one button press	Managed EIO warning may still be latched	Capture logs; do not relock; assess seccfg before UT
Red screen and repeated loop even with stock images	Slot metadata or persistent seccfg state	Normalize slot; then last-resort seccfg diagnosis
Signature/bad-state screen while locked	Modified chain with a locked bootloader	Re-unlock; restore stock vbmeta; never relock modified chain
Both slots say unbootable	A/B metadata exhausted	Restore matched images, reset/select slot A
Installer flashes all three bootstrap images and recovery starts	Android chain has been replaced coherently	Let installer finish; do not interrupt

Part III - What Ubuntu Touch is on this device

Ubuntu Touch, Halium, and the Rabbit port

12.1 Ubuntu Touch is not desktop Ubuntu copied onto a phone

Ubuntu Touch combines several layers:

```
+-----+
| Lomiri shell, Ubuntu applications, system services |
| Ubuntu Touch root filesystem                       |
+-----+
| Halium adaptation layer                             |
| libhybris and Android-compatible hardware services |
+-----+
| Device-specific kernel, init, SELinux/AppArmor work |
+-----+
| Proprietary vendor blobs from the Android base     |
| GPU, camera, audio, modem, sensors, Wi-Fi, Bluetooth |
+-----+
| Rabbit R1 hardware                                 |
+-----+
```

The Rabbit port is identified as `r1` and uses Halium 12.0. Halium provides a common adaptation surface so an Ubuntu userspace can use an Android-derived kernel and vendor interfaces. Porting is therefore not equivalent to booting a generic ARM64 Ubuntu root filesystem; the kernel, ramdisk, hardware services, vendor libraries, permissions, and display stack must match the device.¹

12.2 Why stock rabbitOS v0.8 is a prerequisite

The official Rabbit R1 Ubuntu Touch page requires:

1. the Rabbit R1 model;
2. stock rabbitOS v0.8 firmware;

¹UBports' porting documentation explains the Ubuntu Touch root filesystem, Halium adaptation, Android-derived kernel, and vendor blob relationship. See source note 9.

3. an unlocked bootloader.²

The stock requirement supplies the expected non-super firmware and hardware interfaces: bootloader, modem firmware, trusted execution components, device-tree data, and other vendor-specific partitions. The Ubuntu Touch bootstrap replaces the main boot, vbmeta, and dynamic system image, but it does not reconstruct every low-level MediaTek partition from scratch.

This is why restoring a coherent v0.8.293 base before installation is more reliable than continuing from a partially modified Android state.

12.3 Current support status and realistic expectations

At the research date, the official device page describes the port as booting and running well for advanced users and lists working camera, Wi-Fi, Bluetooth, audio, touchscreen, charging, sensors, and Waydroid support. USB MTP and ADB are marked partial. It also lists hardware limitations including no incoming/outgoing calls, no VoLTE, no NFC, no automatic brightness, no flashlight, and no wired external display.³

The channel information is in transition. The device page currently advertises Ubuntu Touch 20.04 OTA-12 as its stable channel, while UBports' official Ubuntu Touch 24.04-1.0 release announcement explicitly includes Rabbit R1, and the 24.04-2.0 test program also included it.⁴⁵⁶

For a fresh installation, the correct rule is:

Select the **stable channel actually offered for Rabbit R1 by the current UBports Installer**. Do not manually force a daily, release-candidate, or development channel merely because a newer platform release exists.

Device-page and release metadata can lag one another. The installer queries the image service and is the operational source of truth for the available channel at install time.

12.4 No supported dual boot

UBports explicitly discourages manual installation when an installer configuration exists and states that dual boot is not supported.⁷ Installing Ubuntu Touch should be treated as replacing the Android userspace on the active device, not adding a selectable second operating system.

²The official Rabbit R1 device page lists Halium 12.0, the UBports Installer, stock rabbitOS v0.8, and an unlocked bootloader as the installation prerequisites. See source note 7.

³The official device page reports its current feature matrix, partial USB status, advanced-user classification, and unsupported hardware features. See source note 7.

⁴The official Rabbit R1 device page lists Halium 12.0, the UBports Installer, stock rabbitOS v0.8, and an unlocked bootloader as the installation prerequisites. See source note 7.

⁵UBports' Ubuntu Touch 24.04-1.0 release announcement includes Rabbit R1 among the supported devices. See source note 14.

⁶UBports' Ubuntu Touch 24.04-2.0 test announcement includes Rabbit R1. At the research date, current stable availability should be taken from the installer. See source note 15.

⁷UBports' supported-devices guidance says dual boot is unsupported and manual installation is discouraged when the installer is available. See <https://devices.ubuntu-touch.io/>.

What the UBports Installer actually does

The Rabbit R1 installer configuration is unusually useful because it makes the bootstrap sequence explicit.¹

13.1 Device-specific button actions

The configuration defines:

- **Bootloader:** power off, move the scroll wheel up while pressing power; if a menu appears, select Fastboot.
- **Recovery:** power off, move the scroll wheel down while pressing power.

Your preloader serial method remains a valid recovery entrance when the button route is unreliable:

```
"$R1ESC/.venv/bin/python" "$R1ESC/mtkbootcmd.py" FASTBOOT  
fastboot devices -l
```

13.2 Installer prerequisites

The configuration asks the user to confirm stock rabbitOS v0.8 and an unlocked bootloader. It can invoke the standardized `fastboot flashing unlock` handler when it detects a locked bootloader, but in your case the unlock is already complete.

The installer requires a sufficiently recent installer application and exposes two important options:

- **Wipe Userdata:** required when switching from Android;
- **Bootstrap:** flash system partitions with fastboot; enabled by default.

¹The official UBports installer configuration for r1 requires stock rabbitOS v0.8 and an unlocked bootloader, downloads checksum-pinned boot/super/vbmeta artifacts, flashes those physical partitions, formats userdata, and re-boots to UBports recovery. See source note 8.

For your device state, both should be enabled.

13.3 Downloaded bootstrap images

The configuration downloads three checksum-pinned artifacts:

1. a Rabbit port `super.zip`, unpacked to `super.img`;
2. a Rabbit port `boot.img`;
3. a known `vbmeta.img` from Google's GSI resources.

The exact URLs and SHA-256 sums are maintained in the installer configuration. This is materially safer than manually mixing files found in multiple community guides.

13.4 Flash order

The bootstrap sequence is:

```
reboot or ask user to enter bootloader fastboot
    |
    v
flash vbmeta.img
    |
    v
flash boot.img
    |
    v
flash complete super.img
    |
    v
format userdata if Wipe Userdata is selected
    |
    v
reboot to UBports recovery
    |
    v
install the selected Ubuntu Touch system image
```

There is no `fastboot reboot-fastboot` step in this device configuration. The complete port `super.img` is flashed as a physical factory image from bootloader fastboot, which is consistent with Android's documented factory-image support for dynamic-partition devices.²

²AOSP documents that `super` contains dynamic-partition metadata and logical partitions, and that a factory `super.img` bundles `system/vendor/etc.` and may be flashed directly without `fastbootd`. See source note 5.

13.5 Why this can repair an ordinary mismatch

A successful bootstrap writes a mutually intended trio:

- the port's kernel and ramdisk in `boot`;
- the port's AVB policy in `vbmeta`;
- the port's logical system/vendor/product layout in `super`.

That removes the most common source of dm-verity errors: a boot image from one build, meta-data from another, and filesystems from a third. It cannot necessarily clear a MediaTek-specific persistent `seccfg` error state, which is why the stock-restore and `seccfg` decision gate remain relevant.

Part IV - Recovery and installation run-book

Safety preparation

15.1 Stop conditions

Stop immediately and reassess if any of the following occurs:

- the firmware directory does not exactly match the expected Rabbit release;
- a command names a partition that is absent from `fastboot getvar all` OR `mtkclient printgpt`;
- `fastboot` reports a locked state after you believed unlock succeeded;
- the device disconnects during a write and does not return to preloader or fastboot;
- a candidate `seccfg` image has a different size from the original partition dump;
- a `seccfg` candidate does not show the expected lock and verity fields before writing;
- the device begins booting while a low-level recovery script is still running.

Do not improvise around those conditions.

15.2 Host checklist

Use a native x86-64 Linux host, a short data-capable USB cable, and a direct USB port. Avoid an unpowered hub. Charge the R1 before beginning. Close serial terminals, modem-manager probes, and duplicate `mtkclient` processes.

Install the host tools from the distribution or Google's current platform-tools package:

```
# Debian/Ubuntu example
sudo apt update
sudo apt install android-sdk-platform-tools-common adb fastboot \
    python3-venv python3-pip git unzip e2fsprogs usbutils
```

Package names vary. The important binaries are:

```
command -v fastboot
command -v adb
command -v python3
fastboot --version
```

15.3 Use one Python environment

```
R1ROOT="$HOME/code/others/rabbit-r1"
R1ESC="$R1ROOT/r1_escape"

cd "$R1ESC"
python3 -m venv .venv
"$R1ESC/.venv/bin/python" -m pip install --upgrade pip wheel
"$R1ESC/.venv/bin/python" -m pip install pyserial
"$R1ESC/.venv/bin/python" -m pip install -r "$R1ESC/mtkclient/requirements.txt"
```

Test imports explicitly:

```
"$R1ESC/.venv/bin/python" - <<'PY'
import serial
import usb.core
print("pyserial and pyusb are available")
PY
```

For `mtkclient`, prefer `udev` rules that allow normal-user USB access. If root is unavoidable, use the exact virtual-environment interpreter:

```
sudo "$R1ESC/.venv/bin/python" "$R1ESC/mtkclient/mtk" printgpt
```

Do not use bare `sudo python3`.

15.4 Preserve the firmware archive and hashes

Your extracted stock directory is:

```
rabbit_OS_v0.8.293_20250516110545/
```

Set a variable and inventory it:

```
FW="$R1ROOT/rabbit_OS_v0.8.293_20250516110545"

find "$FW" -maxdepth 1 -type f -printf '%f\n' | sort
sha256sum "$FW"/* > "$R1ROOT/rabbitOS-v0.8.293-files.sha256"
```

Keep the original ZIP as well. Do not rename files to satisfy a command copied from a different firmware package. For example, the community recovery writeup mentions `super_sparse_backup.img`, while your archive lists `super.img`. Use the official flasher, which reads the package's actual scatter and filenames, rather than substituting one for the other.

15.5 Back up critical persistent partitions

This step uses raw MediaTek access and should be read-only. First print the partition table:

```
cd "$R1ESC/mtkclient"
sudo "$R1ESC/.venv/bin/python" ./mtk printgpt | tee "$R1R00T/mtk-gpt.txt"
```

Only for partition names actually present, create a backup directory and read them one at a time:

```
mkdir -p "$R1R00T/r1-backup"

sudo "$R1ESC/.venv/bin/python" ./mtk r seccfg \
"$R1R00T/r1-backup/seccfg-original.bin"

sudo "$R1ESC/.venv/bin/python" ./mtk r frp \
"$R1R00T/r1-backup/frp-current.bin"
```

If the GPT includes `nvr`, `nvd`, `proinfo`, `protect1`, or `protect2`, preserve them as well, but do not erase or write them. Calculate hashes and make an offline copy:

```
sha256sum "$R1R00T/r1-backup"/* > "$R1R00T/r1-backup/SHA256SUMS"
```

The exact connection timing is the same as your successful FRP dump: start the read command, power the R1 off, and reconnect it in preloader/BROM mode when prompted.

15.6 Enter bootloader fastboot cleanly

Use only one transition method at a time.

Button method:

```
Power off -> scroll wheel up + power -> select Fastboot if prompted
```

Preloader serial method:

```
sudo modprobe cdc_acm
"$R1ESC/.venv/bin/python" "$R1ESC/mtkbootcmd.py" FASTBOOT
fastboot devices -l
```

Do not start another watcher after `fastboot devices` already lists the R1.

15.7 Capture the preflight state

Run the diagnostic block from Chapter 8 and save it. Confirm at minimum:

```
unlocked: yes                or an equivalent true value
slot-count: 2
is-userspace: no             for bootloader fastboot
```

If fastboot still rejects writes as locked, do not proceed. Re-check the FRP backup and unlock state rather than flashing around the policy.

Phase 1 - Restore a coherent stock base

16.1 Preferred method: Rabbit's official WebUSB flash tool

Rabbit publishes an official browser-based flash tool and instructs users to download the latest firmware ZIP, enter preloader fastboot, select the fastboot device, and choose **Flash Stock ROM**.¹

Use a Chromium-family browser with WebUSB support:

1. Keep the bootloader unlocked.
2. Open the official Rabbit R1 Flash Tool.
3. Power off and disconnect the R1.
4. Choose **Enter Fastboot Mode**.
5. Connect the R1 and quickly select the MT65xx Preloader device.
6. When the R1 displays fastboot, choose **Select Device in Fastboot**.
7. Select the extracted `rabbit_os_v0.8.293_20250516110545` folder.
8. Choose **Flash Stock ROM** and allow the complete operation to finish.
9. Do **not** relock afterward.

This method is preferred because the tool and scatter file determine the intended partition set. It reduces the risk of omitting a boot-chain partition or applying flags intended for a GSI.

16.2 What a stock restore should and should not do

A correct stock restore should write a matched set from one release, including the boot chain and super. It should use the stock vbmeta images without `--disable-verity` or `--disable-verification` rewriting.

It should not:

- erase identity/calibration partitions;
- relock the bootloader;

¹Rabbit's official WebUSB tool instructs users to download the firmware release, enter preloader fastboot, select the fastboot device, and flash the stock ROM folder. See source notes 10 and 11.

- use files from CipherOS, a GSI, or Ubuntu Touch in the same pass;
- substitute `preloader_raw.img` for another preloader format without knowing the official flasher's selection logic.

16.3 Manual restore is a fallback, not the first choice

The community recovery describes a manual full pass that flashes the A-slot boot-chain set, a complete shared `super`, stock `userdata`, mirrors boot-chain images to B, and resets slot metadata.² That sequence was written for a particular extracted package whose filenames differ from yours.

If the official tool cannot be used, derive the manual command list from **your** `MT6765_Android_scatter.xml` or `.txt`, not from filenames in another archive. At minimum:

- inspect every partition name and `is_download` setting;
- verify each file exists;
- use the stock `vbmata` images without disable flags;
- flash the complete `super.img` to `super`, not to `system`;
- do not write a preloader image unless the exact storage type and format are confirmed.

A generic-looking command can still be wrong for a specific scatter. This is the point where using Rabbit's official tool is materially safer.

²The community recovery provides a package-specific full stock flash and Rabbit-specific slot metadata reset. It also warns that archive filenames and protected partitions matter. See source note 13.

Phase 2 - Normalize A/B state

17.1 Inspect before erasing metadata

After the stock flash, re-enter bootloader fastboot and run:

```
for v in \  
  current-slot \  
  slot-successful:a slot-unbootable:a slot-retry-count:a \  
  slot-successful:b slot-unbootable:b slot-retry-count:b; do  
  fastboot getvar "$v"  
done 2>&1 | tee "$R1R00T/post-stock-slot-state.txt"
```

Then select A:

```
fastboot set_active a  
fastboot reboot bootloader
```

Catch fastboot again and re-check the variables. AOSP specifies that selecting a slot should clear its unbootable flag and reset its retry count, although vendor implementations can have additional metadata behavior.¹

17.2 Rabbit-specific metadata reset

The community recovery found that the R1 sometimes needed:

```
fastboot erase boot_para  
fastboot erase para  
fastboot reboot bootloader  
# Re-enter fastboot if necessary  
fastboot set_active a
```

Treat this as a device-specific escalation, not a routine first command. Use it only after a complete stock image set is present and after saving the pre-reset slot variables. These partitions

¹AOSP documents default-current-slot fastboot behavior and the slot variables for successful, unbootable, and retry state. See source note 4.

affect boot-control metadata; erasing them while images are still mismatched can make diagnosis harder.²

17.3 Attempt one stock boot

```
fastboot reboot
```

Expected outcomes:

- **Orange warning, then stock setup:** success. The bootloader is unlocked, but the stock chain is coherent.
- **Red warning that continues after a single side-button press:** capture the exact behavior. If stock then reaches setup, do not relock; the persistent warning state still deserves attention before changing OS.
- **Red warning followed by a loop or preloader re-enumeration:** proceed to the `seccfg` decision gate.
- **Signature/bad-state message:** the device is locked or the flashed vbmeta/boot chain is not truly stock. Return to fastboot and resolve that inconsistency first.

²The community recovery provides a package-specific full stock flash and Rabbit-specific slot metadata reset. It also warns that archive filenames and protected partitions matter. See source note 13.

Phase 3 - The seccfg decision gate

18.1 Enter this phase only when all conditions are true

Do not modify `seccfg` unless:

1. the bootloader remains unlocked;
2. a complete, matched stock v0.8.293 set has just been flashed;
3. slot A has been selected and is not marked unbootable;
4. the same red dm-verity/EIO behavior persists;
5. the original `seccfg` has been dumped and hashed;
6. you can reproduce and inspect the community procedure against the exact `mtkclient` source revision you are using.

If any condition is false, go back rather than guessing.

18.2 What the community recovery found

On the recovered device, the V4 `seccfg` header contained:

offset 0x00	magic	"MMMM"
offset 0x04	version	4
offset 0x08	size	0x3c
offset 0x0c	lock_state	3 (unlocked)
offset 0x10	critical/dm-verity	1 (latched error)
offset 0x14	sboot_runtime	0
offset 0x18	end marker	"EEEE"

The recovery kept `lock_state = 3`, generated a valid authenticated V4 header with the offset 0x10 field set to 0, and cleared the persistent managed-verity record at 0x240:0x2c0. After that, the device booted.¹

¹The community recovery reports a Rabbit-specific persistent dm-verity state in V4 `seccfg`, with lock state at 0x0c, a critical/dm-verity field at 0x10, and a managed-verity record at 0x240:0x2c0. See source note 13.

18.3 Why a hex editor alone is unsafe

The V4 header includes authenticated/encrypted state. Directly changing byte 0x10 in an existing partition dump can invalidate the header integrity data. The community method patched mtkclient's V4 seccfg generator so it produced a correctly authenticated unlocked header with the critical state set to zero. It then combined that generated header with the rest of the device's own full partition dump.

The safe invariant is:

```
same device's full seccfg body
+ valid generated V4 header
+ lock_state still unlocked
+ critical/dm-verity state zero
+ managed-verity record zero
+ unchanged total partition size
```

18.4 Required verification before any write

Inspect the backup first:

```
python3 - <<'PY'
from pathlib import Path
p = Path("r1-backup/seccfg-original.bin")
b = p.read_bytes()
print("size:", len(b))
print("magic:", b[0:4])
print("version:", int.from_bytes(b[0x04:0x08], "little"))
print("header-size-field:", int.from_bytes(b[0x08:0x0c], "little"))
print("lock_state:", int.from_bytes(b[0x0c:0x10], "little"))
print("critical_or_verity:", int.from_bytes(b[0x10:0x14], "little"))
print("managed_record_nonzero:", any(b[0x240:0x2c0]))
PY
```

A candidate must be checked with the same script and with hashes:

```
sha256sum r1-backup/seccfg-original.bin candidate-seccfg.bin
cmp -l r1-backup/seccfg-original.bin candidate-seccfg.bin | head -100
```

Differences should be explainable by the generated header and the managed-record clearing. The candidate length must exactly equal the original length.

18.5 Use the source procedure, not an abbreviated copy

The community writeup includes the two mtkclient patches, environment variables used to generate the 512-byte header, the merge script, expected field values, and the final write command.²

²The community writeup includes the patched mtkclient V4 header generation, full-partition merge, field verification, and write sequence. See source note 13.

Because those patches target code that can change between `mtkclient` revisions, this book intentionally does not present the raw write step as a blind copy-and-paste recipe.

When executing it:

- pin the repository commit;
- save `git diff` showing only the intended changes;
- generate the candidate before writing;
- verify `lock_state = 3`;
- verify the offset `0x10` value is `0`;
- verify the managed record is all zero;
- write once;
- stop all tools if the camera moves or the UI starts.

A late write error does not automatically mean the beginning of the partition was not written, but it is also not permission to retry repeatedly. Observe the device and compare a fresh read-back before another write.

Phase 4 - Install Ubuntu Touch

19.1 Establish the prerequisite state

The ideal starting point is:

```
Bootloader: unlocked
Firmware base: complete stock rabbitOS v0.8.293
Active slot: A, bootable
Screen on stock boot: orange unlock warning only, then setup
USB: bootloader fastboot visible to normal user
Backups: seccfg, frp, and calibration partitions stored offline
```

If stock setup is not usable but a matched stock set is present and the red EIO state has been cleared, the installer may still succeed from bootloader fastboot. The critical requirement is a coherent low-level v0.8 base, not preserving Rabbit user data.

19.2 Obtain and run the official installer

Download the Linux package linked from the official Rabbit R1 Ubuntu Touch device page. Use the current .deb or AppImage for x86-64. Run the application as your normal desktop user; UBports troubleshooting guidance generally expects normal-user USB access rather than launching the whole GUI with `sudo`.¹

For an AppImage, the pattern is:

```
chmod +x ubports-installer-*.AppImage
./ubports-installer-*.AppImage
```

If the application cannot see fastboot, fix udev permissions rather than running an untrusted GUI as root. A quick distinction is:

```
fastboot devices -l
sudo fastboot devices -l
```

¹Official installer links and platform packages are published from the Rabbit R1 device page. UBports troubleshooting documentation should be consulted for current udev and installer details. See source note 7.

If only the second command sees the device, install appropriate Android udev rules, reconnect, and re-login or reload rules.

19.3 Installer selections for this device

Choose:

```
Device: rabbit r1 (codename r1)
Channel: stable, as offered by the installer
Wipe Userdata: enabled
Bootstrap: enabled
```

Confirm stock rabbitOS v0.8 and unlocked bootloader when prompted.

Do not choose:

- an unrelated generic Android device;
- a manual GSI image;
- devel or rc solely to obtain a larger version number;
- no-wipe when switching from Android;
- no-bootstrap while the existing boot/super/vbmeta state is damaged.

19.4 During the bootstrap

Keep a separate terminal available for passive observation:

```
watch -n 0.5 'fastboot devices -l; adb devices -l'
```

Do not issue competing flash or reboot commands while the installer is active.

If the installer asks for bootloader mode:

```
Power off -> scroll wheel up + power -> choose Fastboot
```

If that route fails and the installer is waiting, close any serial watchers and use the preloader command once, then return control to the installer.

The expected high-level progress is:

```
download and verify artifacts
unpack super.zip
flash vbmeta
flash boot
flash super
format userdata
reboot recovery
push/install Ubuntu Touch image
reboot
```


A temporary USB disappearance between those stages is normal. A repeated red dm-verity screen immediately after the newly flashed Ubuntu boot chain is not normal and points back to an uncleared persistent state or an interrupted/mismatched bootstrap.

19.5 First boot

An orange unlocked-bootloader warning is expected. Do not interpret it as a failed Ubuntu installation. Continue past it according to the on-screen prompt. The first system startup performs initialization that subsequent boots do not repeat; avoid power cycling merely because the screen pauses during that phase.

Leave the bootloader unlocked. Relocking would make the device demand a boot chain signed by the configured trusted key and can return it to a signature/bad-state failure.

Phase 5 - Validate the installation

20.1 Basic functional checks

After the Ubuntu Touch setup UI appears, check in this order:

1. touchscreen and rotation;
2. Wi-Fi association and DNS;
3. speaker, microphone, and volume keys;
4. charging and battery reporting;
5. camera photo/video and camera rotation;
6. Bluetooth;
7. mobile data or SMS only if a SIM is installed and the port's current status supports the intended function;
8. reboot and shutdown;
9. recovery entry;
10. system update channel.

Record the OS build shown in System Settings before changing channels.

20.2 Expected limitations

Do not use a missing feature as evidence of a corrupt flash when the official device page marks it unsupported or partial. In particular, Rabbit R1 is not expected to provide conventional voice-call or VoLTE functionality under this port, and MTP/ADB may be partial. The device page classifies the port for advanced users rather than as a fully polished mainstream phone.¹

¹The official device page reports its current feature matrix, partial USB status, advanced-user classification, and unsupported hardware features. See source note 7.

20.3 Preserve a recovery notebook

Save:

- Rabbit stock ZIP and its hash;
- extracted firmware hash list;
- original `seccfg` and FRP backups;
- the pre- and post-stock fastboot variable snapshots;
- UBports Installer log;
- installed Ubuntu Touch channel and build;
- any `mtkclient` commit hash and patches used.

This turns future recovery from guesswork into a reproducible state transition.

Phase 6 - Return to stock if needed

21.1 Preferred rollback

1. Back up Ubuntu Touch user data that can be exported.
2. Power off the R1.
3. Enter preloader fastboot using the official Rabbit tool or `mtkbootcmd.py`.
4. Use Rabbit's official flash tool with the complete v0.8.293 folder.
5. Select slot A if needed and boot stock.
6. Keep the bootloader unlocked until stock has booted successfully and all stock vbmeta images are confirmed.

Relocking is not required for stock to operate and should not be part of troubleshooting. Only consider it if Rabbit provides a documented, supported relock procedure for an entirely stock verified chain and you accept the possibility of another data wipe or boot failure.

21.2 Why `gsi wipe` is not the rollback plan

The Ubuntu Touch installer flashes a complete port `super.img` and boot image, not merely a temporary GSI layer. Use the complete stock firmware restore rather than relying on `fastboot gsi wipe`, whose scope is limited to Android's GSI mechanisms.

Part V - Diagnostic reasoning and field reference

The recovery decision tree

```

START: R1 reaches preloader or bootloader fastboot
|
+-- Does `fastboot devices` list it?
|   |
|   +-- NO -> load cdc_acm, send FASTBOOT once, fix USB permissions
|   |
|   +-- YES
|       |
|       +-- Does fastboot permit partition writes?
|           |
|           +-- NO -> bootloader policy still locked
|               |
|               verify FRP/unlock flow; do not flash around it
|           |
|           +-- YES
|               |
|               +-- Backups and state snapshot complete?
|                   |
|                   +-- NO -> dump seccfg/FRP and record slots
|                   |
|                   +-- YES
|                       |
|                       +-- Full matched stock v0.8.293 flash
|                           |
|                           +-- Stock boots after orange warning
|                               |
|                               |
|                               +-- Run UBports Installer
|                                   |
|                                   Bootstrap + Wipe + stable
|                               |
|                               +-- Stock still red/loops
|                                   |
|                                   +-- Select/reset slot A
|                                       |
|                                       +-- Still red/loops?
|                                           |
|                                           +-- NO -> Installer
|                                           |
|                                           +-- YES -> seccfg gate
|                                               |
|                                               backup, generate,
|                                               inspect, write once
|
+-- Ubuntu bootstrap flashes boot/vbmeta/super successfully?

```

```
|
+-- NO -> preserve installer log; restore stock; fix USB/flash error
|
+-- YES
    |
    +-- UBports recovery starts?
        |
        +-- NO -> persistent boot state or interrupted bootstrap
        |
        +-- YES -> let system-image install finish
```

May the installer be tried immediately?

Technically, a single direct UBports Installer attempt can repair an ordinary boot/vbmeta/super mismatch because its bootstrap overwrites those three components with a matched set. Your log also shows that the bootloader is unlocked and reachable.

However, the official Rabbit port requires a stock v0.8 firmware base, and your current state is only partly known. The conservative route is therefore to restore stock first. A direct attempt is reasonable only when all of these are true:

- the low-level partitions are already from v0.8.293;
- `fastboot` is unlocked and stable;
- critical persistent partitions have been backed up;
- no manual `seccfg` write has been made;
- you accept that a failed attempt may still require the full stock restore.

If you take the shortcut, select **Bootstrap** and **Wipe Userdata**. If the installer flashes its images but the same red EIO loop remains, stop after that one attempt. Repeatedly alternating stock and Ubuntu images will consume slot retries and obscure the persistent-state diagnosis.

Installer-stage troubleshooting

Installer stage	Symptom	Interpretation	Controlled response
Device detection	Installer sees no R1	udev permissions or wrong USB mode	Verify fastboot devices -l as normal user; re-enter bootloader
Unlock check	Installer says locked	LK still reports locked	Stop; verify fastboot getvar unlocked; do not bypass with vbmeta flags
Download	Hash or network failure	Artifact was not verified	Retry download; do not use the partial file
Flash vbmeta/boot	not allowed in locked state	unlock was lost or misdetected	Return to unlock diagnosis
Flash super	transfer/write failure	cable, host USB, image, or eMMC problem	Save log; reconnect to bootloader; do not flash random logical partitions
Format userdata	format failure	host tooling or filesystem issue	Let installer retry once; use current platform-tools/e2fsprogs; avoid repeated -w
Reboot recovery	device returns to preloader	boot/recovery did not start	Re-enter bootloader, verify bootstrap completed, assess persistent EIO state
System image install	ADB/recovery lost	recovery USB or interrupted install	Re-enter UBports recovery with wheel down + power; preserve installer log
First Ubuntu boot First Ubuntu boot	orange warning red dm-verity warning and loop	unlocked bootloader persistent EIO or interrupted/mismatched bootstrap	expected; continue stock restore, slot check, seccfg gate

Why common alternative approaches fail

26.1 Flashing only `boot.img`

`boot.img` supplies the kernel and ramdisk, but its AVB hash and its expected userspace must match `vbmeta` and `super`. A new boot image against an old system may fail before or during first-stage init.

26.2 Flashing only `vbmeta.img`

A disabled `vbmeta` can suppress one layer of verification, but it does not install a compatible kernel or userspace. It can also leave chained `vbmeta` partitions or persistent managed-verity state unchanged.

26.3 Flashing a GSI `system.img` without a stable `fastbootd`

A logical `system` image requires userspace `fastboot` or another tool that understands the dynamic layout. The R1 was not maintaining `fastbootd` in the shown state. The Ubuntu installer avoids this by flashing a complete `super.img`.

26.4 Relocking to “make verification normal”

Relocking does not repair a modified chain. It changes policy from “modified images allowed” to “only an accepted signed chain may boot.” If the current images are disabled, unsigned, or signed by a different key, relocking converts a recoverable warning state into a signature failure. The community recovery explicitly warns against relocking with disabled `vbmeta` installed.¹

¹The community recovery explicitly recommends official Rabbit tools first, describes raw `seccfg` work as a last resort, and warns not to relock with disabled `vbmeta` installed. See source note 13.

26.5 Erasing everything MediaTek exposes

`mtkclient` can expose partitions that ordinary fastboot intentionally protects. Erasing `nvr`, `nvdata`, `proinfo`, `protect*`, RPMB-related data, or calibration partitions can destroy radio identity, camera tuning, sensor calibration, or factory provisioning. More access is not the same as more safety.

Command semantics reference

27.1 MediaTek and serial commands

27.1.1 `python mtkbootcmd.py FASTBOOT`

Waits for the preloader CDC ACM interface and sends an ASCII boot command. It does not flash, unlock, or verify images.

27.1.2 `mtk printgpt`

Reads the device's partition table through the MediaTek DA path. Use it to verify names and sizes before raw reads.

27.1.3 `mtk r PARTITION FILE`

Reads one raw partition into a host file. Appropriate for backups when the partition name is verified.

27.1.4 `mtk w PARTITION FILE`

Writes one raw partition. This bypasses much of fastboot's safety policy. Reserve it for a reviewed recovery procedure.

27.2 Fastboot state commands

27.2.1 `fastboot devices -l`

Lists bootloader-fastboot or fastbootd devices. An empty result says nothing about preloader/BROM mode.

27.2.2 `fastboot getvar NAME`

Asks the current fastboot implementation for state. Output commonly goes to standard error.

27.2.3 `fastboot flashing get_unlock_ability`

Reports whether the unlock operation may be initiated. It is not a universal substitute for `getvar unlocked after unlock`.

27.2.4 `fastboot flashing unlock`

Starts the standardized unlock flow. It changes bootloader policy and generally wipes data. It does not by itself install an OS or rewrite vbmeta flags.

27.2.5 `fastboot set_active a`

Selects slot A and should reset its A/B bootability metadata. Reboot the bootloader and re-check rather than trusting one line of output.

27.3 Fastboot write commands

27.3.1 `fastboot flash boot_a boot.img`

Writes the slot-A kernel/ramdisk image.

27.3.2 `fastboot flash vbmeta_a vbmeta.img`

Writes the image exactly as supplied.

27.3.3 `fastboot --disable-verity --disable-verification flash vbmeta_a vbmeta.img`

Rewrites vbmeta flags before writing. Use only when the intended installation specifically requires disabled verification and keep the device unlocked.

27.3.4 `fastboot flash super super.img`

Writes a complete factory-style dynamic partition image to the physical `super` partition. This is different from `fastboot flash system system.img`.

27.3.5 `fastboot format userdata`

Asks fastboot to create a filesystem on `userdata`. Host and device implementation details vary.

27.3.6 `fastboot -w`

Composite wipe/format behavior. Read every line; an erase may succeed even if a later host-side format fails.

27.4 Reboot commands

27.4.1 `fastboot reboot`

Leaves fastboot and attempts normal boot.

27.4.2 `fastboot reboot bootloader`

Restarts LK fastboot and is useful after changing slot metadata.

27.4.3 `fastboot reboot-fastboot`

Requests userspace fastbootd. It depends on a viable recovery/boot path and is not required by the official Rabbit Ubuntu Touch installer.

A compact command sheet for your next session

This sheet assumes the repositories and stock firmware are already present at the paths used in your transcript.

28.1 1. Set paths and environment

```
R1ROOT="$HOME/code/others/rabbit-r1"
R1ESC="$R1ROOT/r1_escape"
FW="$R1ROOT/rabbit_OS_v0.8.293_20250516110545"

cd "$R1ESC"
[ -x .venv/bin/python ] || python3 -m venv .venv
.venv/bin/python -m pip install -U pip wheel
.venv/bin/python -m pip install pyserial
.venv/bin/python -m pip install -r mtkclient/requirements.txt
```

28.2 2. Back up persistent state

```
mkdir -p "$R1ROOT/r1-backup"
cd "$R1ESC/mtkclient"

sudo "$R1ESC/.venv/bin/python" ./mtk printgpt \
| tee "$R1ROOT/mtk-gpt.txt"

sudo "$R1ESC/.venv/bin/python" ./mtk r seccfg \
"$R1ROOT/r1-backup/seccfg-original.bin"

sudo "$R1ESC/.venv/bin/python" ./mtk r frp \
"$R1ROOT/r1-backup/frp-current.bin"

sha256sum "$R1ROOT/r1-backup"/*.bin \
> "$R1ROOT/r1-backup/SHA256SUMS"
```

Reconnect the R1 as directed for each preloader operation. Add other calibration partitions only after confirming their exact names in the GPT.

28.3 3. Enter bootloader fastboot and record state

```
sudo modprobe cdc_acm
"$R1ESC/.venv/bin/python" "$R1ESC/mtkbootcmd.py" FASTBOOT
fastboot devices -l

{
  fastboot getvar unlocked
  fastboot getvar current-slot
  fastboot getvar slot-successful:a
  fastboot getvar slot-unbootable:a
  fastboot getvar slot-retry-count:a
  fastboot getvar slot-successful:b
  fastboot getvar slot-unbootable:b
  fastboot getvar slot-retry-count:b
  fastboot getvar is-userspace
} > "$R1R00T/pre-restore-fastboot.txt" 2>&1

cat "$R1R00T/pre-restore-fastboot.txt"
```

28.4 4. Restore stock with the official Rabbit tool

Use the official browser flash tool and select:

```
$FW
```

Do not append disable-verification flags and do not relock.

28.5 5. Select and inspect slot A

```
fastboot set_active a
fastboot reboot bootloader
# Re-enter fastboot if the USB device does not return automatically.

for v in current-slot \
  slot-successful:a slot-unbootable:a slot-retry-count:a \
  slot-successful:b slot-unbootable:b slot-retry-count:b; do
  fastboot getvar "$v"
done 2>&1 | tee "$R1R00T/post-restore-slots.txt"
```

Only if full stock is present and metadata remains bad, apply the Rabbit-specific reset from the community recovery:

```
fastboot erase boot_para
fastboot erase para
fastboot reboot bootloader
```



```
# Re-enter fastboot
fastboot set_active a
```

28.6 6. Test stock

```
fastboot reboot
```

- Orange then setup: continue to installer.
- Red EIO and loop: use the `seccfg` gate; do not relock.

28.7 7. Run UBports Installer

```
rabbit r1
stable channel offered by installer
Wipe Userdata = on
Bootstrap = on
```

Do not run competing fastboot commands during installation.

Concept checks

29.1 Why did fastboot devices initially show nothing even though FASTBOOT was sent?

Because the preloader serial write and bootloader USB enumeration are separate events. The command could be accepted while the device later failed to remain in fastboot, enumerated too briefly, or lacked host permissions.

29.2 Why did changing FRP affect unlock ability?

On this Rabbit firmware, the community script uses a byte in FRP as the persistent OEM-unlock eligibility switch. The transcript's unlock behavior changed immediately after that write. This is device/build-specific and should not be generalized to every Android device.

29.3 Why is a stock `super.img` not the same thing as the GSI's `system.img`?

`system.img` is one logical filesystem image. `super.img` is a physical container image that includes dynamic-partition metadata and multiple logical partitions. Flashing one where the other is expected changes both scope and target.

29.4 Why can an unlocked device still show dm-verity corruption?

Unlocking permits modification; it does not guarantee that the selected images agree or clear every persistent error state. AVB/dm-verity policy and bootloader lock policy are related but distinct.

29.5 Why restore stock before Ubuntu Touch if the installer overwrites the main images?

The port expects a particular low-level firmware base outside `boot`, `vbmeta`, and `super`. A stock restore also gives a diagnostic control: if stock cannot boot, the problem is below or outside the Ubuntu userspace image.

29.6 Why leave the bootloader unlocked after Ubuntu Touch boots?

The Ubuntu Touch image is not the Rabbit factory signed chain expected by a stock locked bootloader. Relocking can make LK reject it before Linux starts.

Glossary

A/B device

A device with two boot slots so updates can be written to one while the other remains a fallback.

ADB

Android Debug Bridge. A userspace protocol available only when a running OS or recovery exposes its daemon and authorizes the host.

Android Verified Boot (AVB)

The signed metadata and verification framework used to authenticate the Android boot chain and read-only partitions.

BROM

Boot ROM. Immutable first-stage code inside the MediaTek SoC.

Bootstrap

In the UBports Installer, the fastboot phase that writes the device-specific boot, vbmeta, and system partition images needed to start UBports recovery.

CDC ACM

A USB serial class used by the R1 preloader and exposed on Linux as `/dev/ttyACM*` through the `cdc_acm` driver.

Chained vbmeta

An AVB arrangement where top-level vbmeta delegates verification of groups such as system or vendor to another signed vbmeta partition.

Download Agent (DA)

MediaTek code uploaded through BROM/preloader to provide storage operations. `mtkclient` uses this path.

dm-linear

Linux device-mapper target used to map extents inside `super` into logical partitions.

dm-verity

Linux device-mapper target that verifies read-only blocks against a Merkle hashtree.

EIO mode

An AVB hashtree error mode represented to Android as `androidboot.veritymode=eio`; the reference boot UI uses a red corruption warning for this state.

fastboot

A flashing protocol. On the R1, bootloader fastboot runs in LK and obeys bootloader lock policy.

fastbootd

Userspace fastboot, normally launched from recovery to manipulate dynamic logical partitions.

FRP partition

Persistent Factory Reset Protection and policy storage. The `r1_escape` script modifies a byte here to enable the Rabbit unlock flow.

GSI

Generic System Image. A generic Android `system.img` intended for Project Treble-compatible devices; it does not include all device-specific firmware.

Halium

Compatibility and adaptation infrastructure that lets Ubuntu Touch use an Android-derived kernel and vendor hardware stack.

LK

Little Kernel, commonly used as a MediaTek Android bootloader stage. It implements fastboot, slot selection, and AVB decisions on this device family.

Logical partition

A virtual block device such as `system_a` whose extents are allocated inside `super`.

Orange state

The expected warning state for an unlocked Android bootloader.

Preloader

MediaTek's early boot stage loaded after BROM; it initializes memory and exposes transient USB protocols.

seccfg

MediaTek security configuration storage. On the community-recovered Rabbit build, it held both lock state and a persistent dm-verity-related state.

Slot metadata

Boot-control information recording active, successful, unbootable, and retry-count state for slots A and B.

super partition

A physical dynamic-partition container holding system, vendor, product, and related logical images.

vbmeta

The AVB metadata image containing signed descriptors, flags, hashes, hashtrees, or chained-partition information.

Selected source notes

The most important operational sources are listed below. Access dates are 2026-08-07.

1. Android Open Source Project, Boot flow.
2. Android Open Source Project, Android Verified Boot.
3. Android Open Source Project, Device state.
4. Android Open Source Project, Implement A/B updates.
5. Android Open Source Project, Implement dynamic partitions.
6. Android AVB source and README, platform/external/avb.
7. UBports, Rabbit R1 device page.
8. UBports, Rabbit R1 installer configuration.
9. UBports documentation, Introduction to porting.
10. Rabbit HMI OSS, Rabbit R1 Flash Tool.
11. Rabbit HMI OSS, Firmware releases.
12. RabbitHoleEscapeR1, r1_escape.
13. Jonathan Procter, Rabbit R1 dm-verity recovery community writeup.
14. UBports, Ubuntu Touch 24.04-1.0 release.
15. UBports, Ubuntu Touch 24.04-2.0 beta announcement.